

Estructura de Computadors II
Curs 2021/2022

PRÀCTICA FINAL:

GAME OF LIFE

Jaume Adrover Fernández	DNI: 43235882E
Joan Balaguer Llagostera	DNI: 43219174N

Índex

Introducció	3
Apartat 1	3
Apartat 2	4
Apartat 3	5
Conclusió	7

1. Introducció

En aquesta pràctica, ens demanaven que implementessim una aplicació amb una interfície gràfica que dugues a terme una versió per a dos jugadors del Joc de la Vida (*Game of Life*) de John Conway. Respecte del joc original, en aquesta pràctica hem hagut d'introduir algunes modificacions:

- La Graella del nostre joc (a diferència de la del joc original) es finita. Això ens permet dividir el taulell finit en dos de més petits, per tal que cada una d'aquestes divisions representi el taulell de cada jugador.
- El joc està adaptat per a 2 jugadors encara que les regles son molt semblants a l'original.

Per dur a terme el joc, s'han implementat 3 apartats, els quals són seqüencial (fins que no tens fet el primer, no pots passar a fer el segon).

2. Apartat 1

En aquest aquest apartat s'han dut a terme les següents subrutines de l'arxiu SYSTEM.X68 (cap d'aquestes, té cap paràmetre d'entrada ni de sortida):

- **SYSINIT:** es la subrutina encarregada de inicialitzar el sistema, primer deshabilitant les interrupcions, després cridant a les subrutines SCRINIT I MOUINIT, habilitant les interrupcions i canviant a mode usuari.
- **MOUINIT:** aquesta subrutina, inicialitza el ratolí, emmagatzemant la posició actual del ratolí i l'estatus actual dels seus botons dins les variables MOUY, MOUX i MOUVAL, fent un clear a la variable MOUEDGE i instal·lant MOUREAD dins el TRAP #MOUTRAP.
- **MOUREAD:** es l'encarregada de, mitjançant una cridada al TRAP 15, enmagatzemem les coordenades actuals del ratolí dins MOUY, MOUX i l'estat dels botons d'aquest a MOUVAL. També enmagatzemam el valor actual de la variable MOUEDGE.

3. Apartat 2

En aquest apartat, hem de implementar subrutines a dos arxius distints: **BUTTON.X68** i **BTNLIST.X68**. Dins els primer, hem hagut d'implementar les funcionalitats bàsiques d'un botó:

- **BTNINIT** (inicialització): en aquesta subrutina, simplement copiam el punter de l'estatic data block (conté la informació que després utilitzarem per poder dibuixar cada un dels botons) dins el variable data block (conté, un punter al SDB i un byte d'informació respecte de l'estat actual del botó, que anomenem status byte). També fem un clear de l'status byte.
- **BTNUPD** (actualització): és la subrutina encarregada de actualitzar els valors dels bits IPC de l'status byte del VDB en funció de les coordenades actuals del ratolí. Per dur-ho a terme, primer necessitem obtenir el valor de les variables **XLEFT**, **YTOP**, **WIDTH** i **HEIGHT**. Una vegada obtingudes, les anem comparant per tal de saber si el ratolí està o no dins el recinte del botó. En funció dels resultats obtinguts, modificarem 1, 2 o 0 bits de l'status byte.
- **BTNPLT** (dibuixat): es l'encarregada d'implementar les funcionalitats per dibuixar un botó. Per fer-ho, primer establim la gruixuda del contorn a la constant **BTNPENWD**, després establim el color amb el qual dibuixarem a la constant **BTNPENCL** i fem les seves corresponent cridades al TRAP 15. Posteriorment, miram si està està o no pitjat el botó, a més de mirar si hi ha el ratolí dins aquest (ho feim mirant els bits IPC de l'status byte) per tal de dibuixar-lo d'u o un altre color tal com ens indica l'enunciat. Finalment, dibuixam el botó i el seu corresponent text (que obtenim de l'SDB contingut al VDB) amb 2 cridades més al TRAP 15.

Dins el segon, es gestionen les funcionalitats bàsiques d'una llista de botons, però nosaltres només hem hagut de programar la que es correspon a dibuixar la llista de botons (la resta ja està programades):

- **BTLPLT**: principalment, aquesta subrutina executa **BTLMXVDB** vegades la subrutina **BTNPLT** per tal d'anar dibuixant tans de botons com ens indiqui aquella constant.

4. Apartat 3

Finalment, a l'últim apartat d'aquesta pràctica, hem hagut de modificar els arxius GOL.X68 i GRID.X68. El primer, s'encarrega, principalment, de gestionar l'aplicació de Game of Life. Concretament s'encarrega de Inicialitzar (GOLINIT), actualitzar (GOLUPD) i dibuixar (GOLPLOT) l'aplicació. També conté algunes subrutines auxiliars i el que s'anomena Static Data. Allà és on hem de definir els callbacks corresponents a cada subrutina. La taula de correspondències entre les diferents etiquetes i les subrutines (callbacks) assignades a cada una d'elles és la següent:

Etiqueta	Subrutina (Callback)
GOLCLRBT	GOLINIT
GOLRUNBT	GOLTORUN
GOLSTPBT	GOLTOPAU
GOLSTEBT	GOLDORUN
GOLLLFBT	GRDLLEFT
GOLLRTBT	GRDLRGT
GOLLOABT	GRDLOAD
GOLSAVBT	GRDSAVE

El segon fitxer, és l'encarregat de gestionar tot allò relacionat amb la graella de joc i el marcador. Dins aquest, hem hagut de programar les següents subrutines:

- **GRDMUPD:** aquesta subrutina s'encarrega d'actualitzar la graella utilitzant les coordenades del mouse. Bàsicament, primer comprova si les coordenades del mouse colisionen amb la zona de la interfície gràfica on es mostra la graella, a més de comprovar si s'està pitjant algun dels botons del mouse. En cas que alguna de les dues condicions anteriors no succeeixi, botem directament al final de la subrutina. En cas que alguna d'aquestes dues condicions es compleixi, passem a la segona par de la subrutina. En aquest punt, si el botó del mouse que està pitjat és l'esquerra, llavors passem a comprovar si la casella a on es situa el mouse pertany al jugador 1 (part

esquerra del taulell) o 2 (part dreta del taulell). En els seus respectius casos, modifiquem el valor de la casella que estem tractant a 1 o 2 respectivament. En canvi, si el botó del mouse que està pitjat és el dret, simplement modificarem el valor de la casella a 0.

- **GRDRUPD:** al nostre parèixer, ha estat la subrutina més costosa de tota la pràctica. Aquesta, és la subrutina encarregada de actualitzar la graella a on es desenvolupa el joc. Aquesta serà actualitzada en funció de les regles del Joc de la Vida. Per tant, cada vegada que s'executi aquesta subrutina, es durà a terme una generació del joc. Primer, afegim un 1 al contador de Generacions GRDNGEN. Després, canviem el punters de les matrius destí i font (GRDDST GRDSRC). En aquest punt de la subrutina, comença el tractament per caselles. Primerament, comprovem quina és la situació dels 8 veïnats que té cada casella. Després, passem a comprovar les normes del joc, i ens trobem amb dos casos generals: la casella està morta o la casella està viva. Si està morta, comprovarem les regles per veure si compleix els requisits per començar a viure i en canvi si està viva, comprovarem els requisits perquè aquesta segueixi amb vida. Aquesta estratègia, la repetirem tantes vegades com caselles té la graella, sigui $GRDWIDTH * GRDHEIGHT$ vegades.
- **GRDLLEFT i GRDLRGT:** aquestes dues subrutines són les encarregades de llegir els arxius .csv i traduir la informació contingut dins aquests a dins la graella del nostre joc. GRDLLEFT, ho fa per a la part esquerra del taulell i GRDLRGT per a la part dreta, mentre que GRDLOAD ho fa per tot el taulell sencer. Les dues que hem hagut de programar nosaltres són còpies exactes de GRDLOAD (que ja venia programada), l'única diferència és que a GRDLLEFT, només mostrem caselles fins a la meitat del taulell. En canvi a GRDLRGT ho fem per l'altre meitat del taulell.
- **GRDPLOT:** aquesta subrutina duu a terme múltiples funcionalitats. La primera és dibuixar la pròpia graella. per fer-ho, primer configuram el llapis amb el TRAP 15 amb la constant CLRDKGRN i posam la gruixada corresponent al llapis per poder dibuixar les caselles. Una vegada ho tenim tot a punt, implementem dos bucles (un per recórrer les files i l'altre per a les columnes) per tal de dibuixar cada una de les caselles de la graella. A més, aquesta subrutina, també ha de dibuixar les puntuacions de cada jugador. A continuació de dibuixar la graella, s'imprimeix la puntuació del jugador 1 per

a la qual fem múltiples cridades al TRAP 15 per: posar el color del llapis a blau, ubicar el cursor i imprimir el contador. En el cas del jugador 2, duem a terme la mateixa estratègia que per al jugador 1 amb la diferència que posarem el llapis per pintar de vermell. Cal mencionar que, per a cada un dels marcadors, es duu a terme una sentència condicional per tal de mirar quin dels dos marcadors de puntuacions (GRDNPLR1 i GRDNPLR2) duu una puntuació més alta. En el cas de la més alta, dibuixem l'asterisc que indica que el jugador va guanyant. Per últim, hem de dibuixar el comptador de generacions, per al qual seguim la mateixa estratègia que per als marcadors dels jugadors, però aquesta vegada posant el seu corresponent llapis.

-

5.Conclusió

En conclusió, hem vist aquesta pràctica com un repte. Consideram que engloba múltiples funcionalitats del llenguatge ensamblador fins ara noves per a nosaltres, desde els gràfics fins a la reproducció de sons o manipulament de fitxers. Ens ha permès repassar tot el vist durant el quadrimestre i plasmar-ho amb el 68K.

Personalment, trobam que ens ha ajudat molt a relacionar conceptes matemàtics vists abans com el mòdul, per tal d'enfocar-ho en l'àmbit de la programació, on es demostra que les matemàtiques són la base fonamental d'aquesta. Malgrat